

V MARATONA PACODE

8 de Novembro de 2025

Caderno de problemas
Instituição: *CI - UFPB*

Instruções

Esta prova contém um total de **9** problemas. As páginas estão numeradas de 1 a 19, contando com essa página de rosto.

A) Sobre os nomes dos programas

- Se sua solução é implementada em C/C++ ou em Python, o nome do arquivo não é significativo, pode ser qualquer nome.
- Se sua solução for implementada em java, é obrigatório que o nome do arquivo seja no formato *Letra.java*, onde *Letra* corresponde à letra que representa o problema. Lembre que o nome da classe principal em java deve ter o mesmo nome do arquivo.

B) Sobre a entrada

- A entrada do seu programa deve ser lida da *entrada padrão*.
- A menos que a própria questão informe a quantidade ou as possíveis quantidades de casos de teste, assuma que esse valor é sempre 1.
- Quando a entrada possuir múltiplos valores, cada par de valores consecutivos numa mesma linha estará separado por exatamente um espaço em branco.

C) Sobre a saída

- A saída de seu programa deve ser escrita na *saída padrão*.
- Quando a saída possuir vários valores, esses valores devem estar separados por não mais que 1 espaço em branco. O último caractere impresso deve, obrigatoriamente, sempre ser o de nova linha (`\n`).



Direção do PACODE: Gabriel Campelo Formiga,
Taylor Klaus Cantalice Nóbrega
Prof. Lincoln David Nery e Silva,
Luerson de Albuquerque Silva Filho,
Luiz Henrique Queiroz de Albuquerque Silva

Professor Orientador: Prof. Lincoln David Nery e Silva

Autoria dos Problemas: Gabriel Campelo Formiga
Luiz Henrique Queiroz de Albuquerque Silva,
Prof. Thiago Gouveia (IFPB)

Revisão e Orientação: Prof. Lincoln David Nery e Silva,
Luerson de Albuquerque Silva Filho

Design do Caderno: Taylor Klaus Cantalice Nóbrega

A. Almoço

Autoria: Gabriel Campelo Formiga

Tempo Limite: 1s

Memória Limite: 256 MB

A longa e torturante espera (quase 30 dias de férias, um absurdo!) terminou. Enfim, um novo semestre começa. É uma nova oportunidade para re-pagar aquelas disciplinas com seus professores favoritos e buscar o lendário CRA 9.99.

A sua rotina no Centro de Informática é "tranquila". Suas aulas começam às 5:00 da manhã (porque acordar cedo é saudável, dizem) e vão até às 20:00.

Mas, para sobreviver a essa jornada, só falta um detalhe: comida. E como um bom estudante do CI, todo o seu precioso e escasso dinheiro é gasto com almoço e salgados do Yao. Os salgados são tão bons que sua estratégia financeira é simples: assim que o mês começa, você compra **um** almoço (para garantir a primeira refeição) e, em um ataque de euforia gastronômica, gasta **todo o resto** do seu dinheiro comprando o máximo de salgados que conseguir.

A pergunta que não quer calar é: depois de garantir o almoço e se entupir de salgados, quanto de dinheiro sobra no final desse seu "planejamento"?

Entrada

A primeira linha da entrada contém três inteiros X ($A \leq X \leq 100000$), A ($1 \leq A \leq 1000$) e S ($1 \leq S \leq 1000$), representando, respectivamente: o seu dinheiro total (o orçamento do mês), o preço de um almoço e o preço de um salgado do Yao.

Saída

Imprima um único inteiro: o que sobrou para você sobreviver o resto do mês (se é que sobrou algo).

Exemplo de entrada 01:	Exemplo de saída 01:
1234 150 100	84

Exemplo de entrada 02:	Exemplo de saída 02:
80114 377 466	51

Exemplo de entrada 03:	Exemplo de saída 03:
2000 1700 500	300

B. Bolas de Ping Pong

Autoria: Gabriel Campelo Formiga

Tempo Limite: 1s

Memória Limite: 256 MB

Professor Ewerton passou as férias treinando Marcus para o torneio internacional Kuawei de Ping Pong. O plano foi bem sucedido e Marcus se tornou um mestre da arte do tênis de mesa. Contudo, motivado pela ganância, ele resolveu criar um jogo com o objetivo de coletar o máximo de bolas possíveis.

Como você bem sabe, toda bola de ping pong é identificada por uma letra minúscula do alfabeto latino. Dada uma sequência de bolas representada por uma string s , Marcus pode realizar a seguinte operação até que sobre apenas uma bola:

- Escolher um índice i ($1 \leq i \leq |s| - 1$, onde $|s|$ denota o tamanho atual de s) tal que $s_i = s_{i+1}$ (ou seja, duas bolas adjacentes são idênticas).
- Em seguida, ele substitui a bola s_i por qualquer outra letra (bola) da sua escolha e remove a bola s_{i+1} da sequência.

Levando em consideração que todas as bolas removidas serão entregues para Marcus, responda qual é o número máximo de bolas que ele consegue coletar.

Entrada

A única linha da entrada contém uma string s ($1 \leq |s| \leq 10000$), composta por letras minúsculas do alfabeto latino, representando a sequência inicial de bolas.

Saída

Imprima um único inteiro, representando o número máximo de bolas que Marcus consegue coletar.

Exemplo de entrada 01:	Exemplo de saída 01:
taa	2

Exemplo de entrada 02:	Exemplo de saída 02:
pacode	0

Exemplo de entrada 03:	Exemplo de saída 03:
petcc	4

C. Copa

Autoria: Gabriel Campelo Formiga

Tempo Limite: 2s

Memória Limite: 256 MB

Duas vezes por ano, o Centro de Informática treme. É a Copa APA. E, como de costume, o Professor Bruck é o responsável por arquitetar “O Problema”, um desafio tão complexo que faz $O(n^3)$ parecer um passeio no parque.

Contudo, a edição 2025.2 foi ambiciosa e o problema de Bruck atingiu níveis absurdos de dificuldade, consumindo todo o seu tempo (e talvez sua sanidade). Em um raro ato de delegação (ou talvez só para se livrar do trabalho braçal), ele convocou seus ajudantes.

Sherts, o mais esforçado (e coitado) deles, pegou a tarefa “fácil”: organizar o pódio da premiação. “Simples”, disse Bruck. Sherts rapidamente descobriu que “simples”, no dicionário de Bruck, significa “um pesadelo combinatório”.

Para montar o pódio, Sherts precisa respeitar regras sagradas de cor e (aparentemente) de ego:

- A base do primeiro lugar deve ser **Vermelha**, a do segundo lugar **Verde** e a do terceiro **Azul**. Sem exceções.
- A altura da base do primeiro lugar (Vermelha) deve ser *estritamente maior* que a altura da base do segundo (Verde), que por sua vez deve ser *estritamente maior* que a do terceiro (Azul).

Por sorte (ou resultado de décadas de compras orçamentárias bizarras), o depósito do CI tem um estoque colossal de bases. Sherts está agora olhando para pilhas gigantescas de peças vermelhas, verdes e azuis, cada uma com uma altura diferente.

Ajude Sherts a calcular o número total de combinações que satisfazem as regras do professor Bruck, antes que ele tenha que testar manualmente todas as peças.

Entrada

A primeira linha da entrada contém três inteiros R , G e B ($1 \leq R, G, B \leq 10^5$), representando o número de bases disponíveis da cor vermelha, verde e azul, respectivamente.

A segunda linha contém R inteiros R_i , representando as alturas das bases vermelhas ($1 \leq R_i \leq 10^9$).

A terceira linha contém G inteiros G_i , representando as alturas das bases verdes ($1 \leq G_i \leq 10^9$).

A quarta linha contém B inteiros B_i , representando as alturas das bases azuis ($1 \leq B_i \leq 10^9$).

Saída

Imprima um único inteiro: o número de formas possíveis de construir o pódio a partir das peças encontradas.

Exemplo de entrada 01:	Exemplo de saída 01:
2 3 5 1 10 5 9 9 3 5 6 6 2	12

Exemplo de entrada 02:	Exemplo de saída 02:
4 2 2 5 5 6 9 4 1 4 9	0

Exemplo de entrada 03:	Exemplo de saída 03:
4 4 3 1 6 10 4 9 3 7 7 6 2 9	9

D. Desafio de Basquete

Autoria: Gabriel Campelo Formiga

Tempo Limite: 2s

Memória Limite: 256 MB

Campelo e Robertinho jogam no time de basquete da maior atlética universitária do Brasil, a Compilada! Robertinho é um garoto exibido e tem como meta de vida enterrar uma bola sobre o maior adversário da Compilada, o temido Elcinho. Para isso, Campelo elaborou um plano e conta com a ajuda da torcida.

No momento oportuno, Campelo fará um passe para Robertinho e gritará: “Faz teu nome, Robertinho!”. Qualquer torcedor a uma distância menor ou igual a D de um grito repassará a mensagem até que todos dentro do raio de alcance a tenham escutado.

Responda se é possível que a mensagem emitida por Campelo chegue aos ouvidos de Robertinho.

Entrada

A primeira linha da entrada contém dois inteiros N ($1 \leq N \leq 2000$), o número de torcedores, e D ($1 \leq D \leq 2000$), a distância máxima em que se pode escutar um grito.

A segunda linha contém dois inteiros X_c, Y_c ($-1000 \leq X_c, Y_c \leq 1000$), representando as coordenadas de Campelo.

A terceira linha contém dois inteiros X_r, Y_r ($-1000 \leq X_r, Y_r \leq 1000$), representando as coordenadas de Robertinho.

As N linhas seguintes contêm, cada uma, dois inteiros X_i, Y_i ($-1000 \leq X_i, Y_i \leq 1000$), representando as coordenadas do i -ésimo torcedor.

Saída

Imprima “Zerei a vida!” se Robertinho consegue escutar a mensagem e “Sobrou Nada!” caso contrário.

Exemplo de entrada 01:	Exemplo de saída 01:
4 10 0 0 25 0 8 0 16 0 24 0 0 20	Zerei a vida!

Exemplo de entrada 02:	Exemplo de saída 02:
3 5 0 0 10 10 4 0 8 0 9 3	Sobrou Nada!

E. Escadas Decrescentes

Autoria: Luiz Henrique Queiroz de Albuquerque Silva

Tempo Limite: 1s

Memória Limite: 256 MB

Bininho é um gato que adora brincar com tudo aquilo que os humanos que moram na sua casa lhe entregam como oferenda. Sofá, poltrona, cama, carpete, cortina, vasos caros, computador deixado desleixadamente bem no cantinho da mesa: nada é deixado sem uso por ele, às vezes fincando suas unhas nas ofertas ou, quando está um pouco entediado, arrastando os pertences (que são dele, claro) para as bordas das mesas, ansiando por descobrir o ponto de equilíbrio - aquele que vem logo antes do objeto cair.

Mas ele reconhece que tem um gosto estranho por escadas decrescentes, ou seja, escadas feitas para você descer. Bob, o servo mais leal de Bininho, decidiu encomendar um código especial para homenageá-lo. O código consiste num programa que recebe um inteiro $|A|$, representando o tamanho de um vetor $a_0, a_1, \dots, a_{n-1}$ que começa com todos os valores iguais a 0, recebendo posteriormente Q *queries* no seguinte formato:

$I V$, sendo I o índice no vetor, e V o valor que deve ser considerado inicialmente;

No índice I , o valor V deve ser adicionado; no índice seguinte $(I + 1)$, o valor $V - 1$, e assim por diante, parando de decrescer o valor ao atingir 0.

Considerando que existe a quase nula (mas não inexistente) possibilidade de que você seja o programador contratado para tão honrosa atividade, se prepare e escreva o código para Bininho. Mãos à obra!

Entrada

A primeira linha da entrada contém dois inteiros $|A|$ e Q : o tamanho do vetor e o número de *queries*.

As próximas Q linhas seguem o formato $I V$: o índice no vetor (começando em 0) e o valor inicial da soma.

Saída

Imprima todo o vetor, com os valores atualizados após as *queries*.

Restrições

$$1 \leq |A|, Q \leq 100$$

$$0 \leq I \leq |A| - 1$$

$$0 \leq V \leq 100$$

Exemplo de entrada 01:	Exemplo de saída 01:
6 1 0 5	5 4 3 2 1 0

Exemplo de entrada 02:	Exemplo de saída 02:
8 2 0 5 6 9	5 4 3 2 1 0 9 8

Exemplo de entrada 03:	Exemplo de saída 03:
10 2 0 10 1 10	10 19 17 15 13 11 9 7 5 3

F. Fermat e a Formiguinha

Autoria: Thiago Gouveia

Tempo Limite: 1s

Memória Limite: 256 MB

Acho que todo mundo já ouviu falar de Fermat, o maior dos matemáticos de tempo livre... Além do seu famoso Último Teorema, Pierre de Fermat também foi responsável por diversos avanços na Matemática, em especial na Teoria dos Números, como o Pequeno Teorema de Fermat, fundamental na criptografia moderna, os estudos sobre a soma de dois quadrados e o método da descida infinita, uma poderosa técnica de prova. Desta vez, ele resolveu estudar o comportamento das formiguinhas em isolamento social (rsrsrsrs). Seu plano é cercar dois grupos de formigas e estudar os movimentos de uma delas, que foi isolada dos grupos. Você poderia ajudar Fermat nesta tarefa? Não que ele precise, é claro, mas ele pode usar esse tempo extra para provar mais um resultado teórico expressivo e fundamental.

São dadas as coordenadas de formiguinhas, e uma delas deve ser escolhida para ser isolada das demais, digamos $F = (x_f, y_f)$. Vamos considerar o grupo L , composto por todas as formigas cuja coordenada x é menor que x_f . Analogamente, o grupo R será composto por todas as formigas cuja coordenada x é maior que x_f .

Para isolar as formigas do grupo L , Fermat usará o menor retângulo (alinhado aos eixos) que contenha todas as formigas desse grupo. Ele fará o mesmo com o grupo R . Como todo matemático tem seus TOCs, ele resolveu que os lados de cada retângulo devem ser números primos! Sua missão é escrever um programa que encontre o menor perímetro (especificamente a soma dos perímetros dos retângulos que cercam os grupos L e R) necessário para isolar uma formiguinha qualquer da coleção de Fermat.

Atenção! Como a formiguinha F deve ser a única isolada, os grupos L e R devem possuir pelo menos duas formigas cada.

Entrada

A primeira linha da entrada traz um inteiro N , representando a quantidade de formiguinhas da coleção de Fermat. Seguem exatamente N linhas, cada uma com dois inteiros, representando as coordenadas de cada uma das formiguinhas de Fermat.

Considere $5 \leq N \leq 10^5$.

Considere que toda coordenada está no intervalo fechado $[1, 2 \times 10^6]$.

Considere ainda que nenhuma coordenada x se repete, assim como nenhuma coordenada y .

Saída

Seu programa deve imprimir apenas um número inteiro positivo: o menor perímetro necessário para isolar uma formiguinha qualquer da coleção de Fermat.

Exemplo de entrada 01:	Exemplo de saída 01:
5 9 2 7 7 2 3 6 4 10 6	28

Exemplo de entrada 02:	Exemplo de saída 02:
12 4 6 9 12 6 2 10 9 8 1 12 5 7 4 1 3 5 8 3 11 2 10 11 7	50

G. Grafos Topologicamente Ordenáveis

Autoria: Luiz Henrique Queiroz de Albuquerque Silva

Tempo Limite: 1s

Memória Limite: 256 MB

Alice estava pobre, lisa, quebrada, sem saber o que dar de presente de aniversário pra Bob; afinal, o que ela poderia oferecer a ele quando todos os seus gostos inevitavelmente fariam ela gastar dinheiro? PC Gamer, livros de Dostoiévski (especificamente Dostoiévski, e aí de quem lhe desse um livro de algum outro autor!), papagaios, iglus, Império Romano: nada disso sairia sem custo.

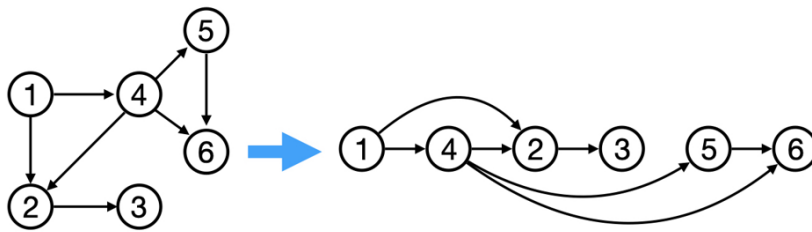
Após muito refletir, não ficou desamparada, pois como que uma iluminação divina lhe atingiu repentinamente e em cheio na sua cabeça quase oca: *Grafos Topologicamente Ordenáveis*. Sim! *Grafos Topologicamente Ordenáveis*! Bob vinha estudando grafos e adorava tudo relacionado a eles: descobrir novos conceitos, desafios de lógica e programação que os envolvesse, e tudo o mais! Portanto, Alice decidiu ensinar a Bob a definição de *Grafos Topologicamente Ordenáveis* e propor um desafio; para tal, entregou a Bob uma carta (que, diga-se de passagem, foi escrita em papel amassado) com o seguinte conteúdo:

Querido Bob, bobinho. Sei o quanto você tem se aventurado no estudo de grafos e, pra complementar o seu estudo e divertimento, trago um conhecimento novo para expandir sua mente: *Grafos Topologicamente Ordenáveis*. Sim, *Grafos Topologicamente Ordenáveis*!

Pense em grafos nos quais os arcos têm direção, saindo de um vértice e indo para outro. Um grafo desses será *Topologicamente Ordenável* se for possível organizar os seus vértices de modo que todos os arcos apontem para a mesma direção e para o mesmo sentido (deixei um exemplo abaixo para você conseguir visualizar melhor; sei que essa sua cabecinha é muito limitada às vezes).

Eu te proponho o seguinte desafio: escrever um programa que, considerando um grafo direcionado de N vértices e o grau de saída - ou seja, quantos arcos “saem” - de cada um deles, informe se é possível que o grafo, em algum universo, seja *Topologicamente Ordenável*. Para facilitar sua vida, ordenei os graus dos vértices em ordem não decrescente de valor.

Com carinho, sua insubstituível amiga, Alice :)



Alice, claro, precisará, de alguma forma, verificar se as respostas de Bob estão corretas. Ajude Alice e crie um programa para resolver o problema também! (Tome o cuidado de fazer certo, por gentileza)

Entrada

A primeira linha da entrada contém um inteiro N : o número de vértices do grafo direcionado.

A segunda linha consiste nos graus de saída $a_0, a_1, \dots, a_{n-1}$ de cada um dos vértices, ordenados de maneira não decrescente de valor.

Saída

Imprima “SIM” (sem aspas) se existe algum grafo direcionado com N vértices e graus de saída especificados que seja *Topologicamente Ordenável*, ou “NAO” (sem aspas) caso contrário.

Nota: considere que grafos sem nenhum arco (todos os graus iguais a 0) são, sim, *Topologicamente Ordenáveis*.

Restrições

$$2 \leq N \leq 100$$

$$0 \leq a_0, a_1, \dots, a_{n-1} \leq N - 1$$

Exemplo de entrada 01:	Exemplo de saída 01:
6 0 0 1 1 2 3	SIM

Perceba que o *Exemplo de entrada 01* é, na verdade, o caso desenhado na carta de Alice.

Exemplo de entrada 02:	Exemplo de saída 02:
2 1 1	NAO

Exemplo de entrada 03:	Exemplo de saída 03:
8 0 1 2 4 4 5 5 5	NAO

Exemplo de entrada 04:	Exemplo de saída 04:
8 0 1 1 3 3 5 6 7	SIM

Exemplo de entrada 05:	Exemplo de saída 05:
3 0 0 2	SIM

Exemplo de entrada 06:	Exemplo de saída 06:
2 0 1	SIM

H. Hortinha de Ortins

Autoria: Gabriel Campelo Formiga

Tempo Limite: 2s

Memória Limite: 256 MB

Depois de longos anos trabalhando para bigtechs, Ortins resolveu deixar a vida de programador para cuidar da sua horta. Seu primeiro desafio é escolher o que plantar em cada um dos seus m lotes de terra. Para isso, Ortins dispõe de n hortaliças distintas numeradas de 1 a n . Além disso, sabe-se que a i -ésima hortaliça possui um custo a_i para ser plantada.

Também, existem k grupos diferentes de hortaliças. Dessa forma, ao analisar a seleção de plantio inicialmente elaborada por Ortins, você pode trocar qualquer hortaliça por outra do mesmo grupo.

Exiba qual é o custo mínimo para Ortins realizar o plantio nos m lotes de terra a partir da sequência inicial.

Entrada

A primeira linha da entrada possui os inteiros n , k e m ($1 \leq k \leq n \leq 10^5, 1 \leq m \leq 10^5$) - o número de hortaliças que podem ser plantadas, o número de grupos e o número de hortaliças na sequência inicial de plantio de Ortins, respectivamente.

A segunda linha contém n strings compostas de letras minúsculas com tamanho menor que 20 representando as diferentes hortaliças. É garantido que nenhuma delas é idêntica.

A terceira linha possui n inteiros $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^9$) representando o custo de plantio da i -ésima hortaliça.

As próximas k linhas descrevem os grupos de hortaliças. Cada uma delas começa com um inteiro x ($1 \leq x \leq n$) que representa a quantidade de elementos do grupo, seguido por x inteiros representando os índices das hortaliças desse grupo. É garantido que cada hortaliça pertence a apenas um único grupo.

A próxima linha contém m strings separadas representando a sequência inicial elaborada por Ortins.

Saída

Exiba um número inteiro representando o menor custo possível para plantar a sequência inicialmente elaborada por Ortins.

Exemplo de entrada 01:	Exemplo de saída 01:
5 4 4 batata couve pepino cenoura alface 100 1 1 5 10 1 1 1 3 2 2 5 1 4 batata pepino cenoura alface	107

Exemplo de entrada 02:	Exemplo de saída 02:
5 4 4 batata couve pepino cenoura alface 100 20 1 5 10 1 1 1 3 2 2 5 1 4 batata pepino cenoura alface	116

I. Inteiros, Primos e Múltiplos

Tempo Limite: 1s

Memória Limite: 256 MB

Você recebe k números primos distintos $a_1, a_2, \dots, a_k$ e um inteiro n . Sua tarefa é calcular quantos dos primeiros n inteiros positivos são divisíveis por pelo menos um dos números primos dados.

Entrada

A primeira linha da entrada contém dois inteiros n ($1 \leq n \leq 10^{18}$,) e k ($1 \leq k \leq 20$).

A segunda linha contém k números primos $a_1, a_2, \dots, a_k$ ($2 \leq a_i \leq n$).

Saída

Imprima um único inteiro: a quantidade de valores no intervalo $1, 2, \dots, n$ que são divisíveis por pelo menos um dos números primos fornecidos.

Exemplo de entrada 01:	Exemplo de saída 01:
20 2 2 5	12

J. Jogando Mais Escadas Decrescentes HAHAAHAHA

Autoria: Luiz Henrique Queiroz de Albuquerque Silva

Tempo Limite: 1s

Memória Limite: 256 MB

Bininho está de mau humor! Deus nos salve!

Para aplacar a fúria de Bininho, Bob decidiu presenteá-lo com um código ainda mais glorioso que o anterior, pois pode lidar com casos teste muito maiores! O código consiste num programa que recebe um inteiro $|A|$, representando o tamanho de um vetor $a_0, a_1, \dots, a_{n-1}$ que começa com todos os valores iguais a 0, recebendo posteriormente Q *queries* no seguinte formato:

$I \ V$, sendo I o índice no vetor, e V o valor que deve ser considerado inicialmente;

No índice I , o valor V deve ser adicionado; no índice seguinte $(I + 1)$, o valor $V - 1$, e assim por diante, parando de decrescer o valor ao atingir 0.

Considerando que existe a quase nula (mas não inexistente) possibilidade de que você seja o programador contratado para tão honrosa atividade, se prepare e escreva o código para Bininho. Mãos à obra!

Entrada

A primeira linha da entrada contém dois inteiros $|A|$ e Q : o tamanho do vetor e o número de *queries*.

As próximas Q linhas seguem o formato $I \ V$: o índice no vetor (começando em 0) e o valor inicial da soma.

Saída

Imprima todo o vetor, com os valores atualizados após as *queries*.

Atenção: use um tipo numérico que suporte valores grandes (como o *long long* em C++, por exemplo).

Restrições

$$1 \leq |A|, Q \leq 10^5$$

$$0 \leq I \leq |A| - 1$$

$$0 \leq V \leq 10^5$$

Exemplo de entrada 01:	Exemplo de saída 01:
6 1 0 5	5 4 3 2 1 0

Exemplo de entrada 02:	Exemplo de saída 02:
8 2 0 5 6 9	5 4 3 2 1 0 9 8

Exemplo de entrada 03:	Exemplo de saída 03:
10 2 0 10 1 10	10 19 17 15 13 11 9 7 5 3

K. kakakakakaka

Tempo Limite: 1s

Memória Limite: 256 MB

Em chats, é muito comum entre jovens e adolescentes utilizar sequências de letras, que parecem muitas vezes aleatórias, para representar risadas. Alguns exemplos comuns são:

huaauhahhuahau
hehehehe
ahahahaha
jaissjksjksjksjakij
huehuehue

Cláudia é uma jovem programadora que ficou intrigada pela sonoridade das “risadas digitais”. Algumas delas ela nem mesmo consegue pronunciar! Mas ela percebeu que algumas delas parecem transmitir melhor o sentimento da risada que outras. A primeira coisa que ela percebeu é que as consoantes não interferem no quanto as risadas digitais influenciam na transmissão do sentimento. A segunda coisa que ela percebeu é que as risadas digitais mais engraçadas são aquelas em que as sequências de vogais são iguais quando lidas na ordem natural (da esquerda para a direita) ou na ordem inversa (da direita para a esquerda), ignorando as consoantes. Por exemplo, “hahaha” e “huaauhahhuahau” estão entre as risadas mais engraçadas, enquanto “riaikjdhhhhjak” e “huehuehue” não estão entre as mais engraçadas.

Cláudia está muito atarefada com a análise estatística das risadas digitais e pediu sua ajuda para escrever um programa que determine, para uma risada digital, se ela é das mais engraçadas ou não.

Entrada

A entrada é composta por uma linha, contendo uma sequência de no máximo 50 caracteres, formada apenas por letras minúsculas sem acentuação. As vogais são as letras ‘a’, ‘e’, ‘i’, ‘o’, ‘u’. A sequência contém pelo menos uma vogal.

Saída

Seu programa deve produzir uma linha contendo um caractere, “S” caso a risada seja das mais engraçadas, ou “N” caso contrário.

Exemplo de entrada 01:	Exemplo de saída 01:
hahaha	S

Exemplo de entrada 02:	Exemplo de saída 02:
riaikjdhhhhjak	N

Exemplo de entrada 03:	Exemplo de saída 03:
a	S

Exemplo de entrada 04:	Exemplo de saída 04:
huaauhahhuahau	S